INDUSTRIAL TRAINING PROGRAM
ROBOTICS AND AI (ROBO AI)

AI-Powered Industrial Chatbot

Using Hugging Face Large Language Models

Field	Details
Project Type	Mini Project — Generative AI
Interface	Streamlit Web Application
Core Model	Ollama LLaMA 3.2
Embeddings	HuggingFace all-MiniLM-L6-v2
Vector Store	FAISS (Facebook AI Similarity Search)
Framework	LangChain + RetrievalQA
Language	Python 3.10+

1. Introduction

This project implements a Retrieval-Augmented Generation (RAG) chatbot. Users upload any PDF document; the system extracts text, builds a semantic vector index, and answers natural-language questions grounded in that document.

1.1 Problem Statement

- Industrial manuals and reports are large — manual search is slow.
- LLMs alone hallucinate without document grounding.
- Solution: combine FAISS semantic retrieval with an LLM for accurate answers.

1.2 Objectives

- Accept PDF uploads through a browser interface.
- Convert document content into a searchable vector index.
- Answer domain-specific questions using retrieved context.
- Deliver responses within the Streamlit chat window.

2. System Architecture

The pipeline follows a classic RAG (Retrieval-Augmented Generation) pattern split into an Ingestion phase and a Query phase.

Phase	Step	Tool / Library
Ingestion	1. PDF Upload	Streamlit file_uploader
Ingestion	2. Text Extraction	PyPDF2 — PdfReader
Ingestion	3. Text Chunking	LangChain RecursiveCharacterTextSplitter
Ingestion	4. Embedding	HuggingFaceEmbeddings (all-MiniLM-L6-v2)
Ingestion	5. Index Storage	FAISS — save_local('faiss_index')
Query	6. Load Index	FAISS — load_local()
Query	7. Retrieve Context	LangChain Retriever (top-k chunks)
Query	8. Generate Answer	Ollama LLaMA 3.2 — RetrievalQA chain
Query	9. Display Answer	Streamlit st.success()

3. Model Selection & Justification

Component	Model / Library	Reason
LLM	Ollama LLaMA 3.2	Runs locally, no API cost, 3B params efficient
Embeddings	all-MiniLM-L6-v2	384-dim, fast, high semantic accuracy
Vector DB	FAISS (faiss-cpu)	Sub-millisecond similarity search
PDF Parser	PyPDF2	Lightweight, no external dependency
UI	Streamlit	Zero-boilerplate web interface

QA Framework	LangChain RetrievalQA	Modular, easy to extend

4. Knowledge Ingestion

4.1 Text Extraction

```
def extract_text_from_pdf(pdf_path):
    reader = PdfReader(pdf_path)
    text = ""
    for page in reader.pages:          # iterate every page
        text += page.extract_text()    # append raw text
    return text
```

Each page's raw text is concatenated into a single string for downstream splitting.

4.2 Chunking Strategy

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,     # ~750 words per chunk
    chunk_overlap=200    # 200-char overlap preserves context at boundaries
)
chunks = splitter.split_text(text)
```

Parameter	Value	Effect
chunk_size	1000	Fits LLM context window comfortably
chunk_overlap	200	Avoids losing context at chunk edges

4.3 Embedding & FAISS Index

```
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
vector_store = FAISS.from_texts(chunks, embedding=embeddings)
vector_store.save_local("faiss_index")   # persisted to disk
```

Each chunk is mapped to a 384-dimensional vector. FAISS builds an approximate nearest-neighbour index enabling fast cosine similarity lookups at query time.

5. Prompt Engineering

LangChain's **stuff** chain type injects retrieved chunks directly into the prompt using the following implicit template:

```
# Effective prompt template (LangChain default for 'stuff' chain):
"""Use the following pieces of context to answer the question.
If you don't know the answer, say you don't know.

Context: {context}

Question: {question}
Answer:"""
```

- Context window: retrieved top-k document chunks substituted as {context}.
- Grounding: the model is instructed to rely only on provided context.
- Fallback: model is told to admit ignorance rather than hallucinate.

6. Q&A; System Implementation

```
def build_qa_chain(vector_store_path="faiss_index"):
    vector_store = load_faiss_vector_store(vector_store_path)
    retriever    = vector_store.as_retriever()       # top-4 chunks by default
    llm          = Ollama(model="llama3.2")          # local LLM
    qa_chain     = load_qa_chain(llm, chain_type="stuff")
    return RetrievalQA(retriever=retriever,
                       combine_documents_chain=qa_chain)
```

At query time:

- User question is embedded with the same MiniLM model.
- FAISS returns the 4 most similar chunks (cosine distance).
- Chunks + question are fed to LLaMA 3.2 inside the stuff prompt.
- The generated answer is displayed via st.success().

7. Chatbot Deployment — Streamlit

```
# Key Streamlit UI flow (app.py)
st.title("AI-Powered Industrial Chatbot ...")
uploaded_file = st.file_uploader("Upload PDF", type="pdf")

if uploaded_file:
    # save → extract → embed → index
    text     = extract_text_from_pdf(pdf_path)
    create_faiss_vector_store(text)
    qa_chain = build_qa_chain()
    st.success("Chatbot is ready!")

question = st.text_input("Ask a question:")
if question:
    answer = qa_chain.run(question)
    st.success(f"Answer: {answer}")
```

Run command: `streamlit run app.py` — opens at http://localhost:8501

8. Installation & Setup

```
# 1. Create virtual environment
python -m venv .venv && source .venv/bin/activate   # Linux/Mac
# .venv\Scripts\activate                           # Windows

# 2. Install dependencies
pip install streamlit PyPDF2 langchain langchain-huggingface
pip install langchain-community faiss-cpu sentence-transformers ollama

# 3. Pull the LLM model (requires Ollama installed)
ollama pull llama3.2

# 4. Run
streamlit run app.py
```

9. Capabilities & Future Scope

Current Capabilities	Future Enhancements
PDF upload via browser	Support DOCX, XLSX, HTML, images (OCR)
Semantic text chunking	Multi-document simultaneous querying

FAISS vector index	Pinecone / Weaviate for cloud scaling
LLaMA 3.2 local inference	Fine-tune on domain corpus
RetrievalQA chain	Chat history & session memory
Streamlit web interface	Auth, rate-limiting, production deploy

10. Conclusion

This project demonstrates a complete RAG-based document QA system built with production-grade open-source tools. The modular design — separate ingestion, retrieval, and generation stages — makes it straightforward to swap components (e.g., replace FAISS with Pinecone, or LLaMA with Mistral) for production scaling. The Streamlit interface makes the system immediately usable without any frontend expertise.